

Chapter 52. The Cloud, Containers & Infrastructure as Code

Cloud fundamentals, a Dockerfile built and validated, Kubernetes concepts, Terraform validated with a real parser, CI/CD trigger logic simulated for real, secrets, and cost.

Draft chapter

19 September 2026

Validated where executable; clearly marked where not

Contents

Chapter 52. The Cloud, Containers & Infrastructure as Code

- Why this matters
- In plain English
- 52.1 Cloud fundamentals
- 52.2 Containers
- 52.3 Orchestrating containers
- 52.4 Infrastructure as Code
- 52.5 CI/CD
- 52.6 Secrets, done properly
- 52.7 Cost
- 52.8 Choosing a deployment target
- Common mistakes and how to spot them
- In the real world: the afternoon the warehouse went dark
- Tools
- The project: deploy Riverstone's pipeline
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 52. The Cloud, Containers & Infrastructure as Code

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: explain regions, availability zones, and the shared responsibility model · design identity and access so a pipeline has exactly the permissions it needs and no more · plan a network with real CIDR arithmetic: VPCs, subnets, and security groups · explain why containers exist and build a Dockerfile for a real pipeline, layer by layer · read a Kubernetes manifest and say what each object is for · write a small Terraform configuration and explain state, plan, and apply · build a CI/CD workflow and trace exactly which jobs run for a given event · keep secrets out of code and images · estimate what running a pipeline in the cloud actually costs · choose a deployment target that fits a company Riverstone's size.

Before you start: Chapter 46 (the pipeline this chapter deploys), Chapter 47 (contracts and incidents), Chapter 49 (storage and cost), Chapter 51 (secrets the pipeline needs).

Time needed: 12–16 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 with `pyyaml`, `python-hcl2`, and `dockerfile-parse`. No Docker daemon, Kubernetes cluster, Terraform provider registry, or cloud account is required, or used, in this chapter's practice environment: see the note below.

A note on what actually runs in this chapter. Building a container image, applying Kubernetes manifests, running Terraform against a real provider, and executing a CI/CD workflow all need services this book's practice environment doesn't have: a container runtime, a cluster, a cloud account, and a hosted repository. Faking their output would be worse than saying so plainly. Instead, every configuration file in this chapter is **real and complete**, and everything that can be checked without those services **is checked for real**: the Dockerfile and Kubernetes YAML are parsed with the same libraries real tools use, the Terraform is parsed with a real HCL parser, and the CI/CD workflow's trigger logic is simulated in Python against the actual YAML, so you can see precisely which jobs would run for a given event. Anything that needs a live daemon, cluster, or cloud account is marked **not executed**, in the same style Chapter 46 used for Airflow and Chapter 50 used for Kafka.

Why this matters

Chapter 46 built a pipeline. Chapter 51 gave it something worth writing to. Right now, both of them exist only as files on your computer, running when you happen to type a command. Getting them to run reliably, on a schedule, reachable by the right people and nobody else, without your laptop being

involved at all, is a different problem, and it's the one this chapter solves.

It's also where a great deal of a data engineer's actual working life happens. Job descriptions ask for "cloud experience" and "containers" more often than they ask for any single analytics tool, not because the cloud is more important than the data, but because nothing you've built in Part V runs anywhere real until it's deployed properly. This chapter takes Riverstone's pipeline from "works on my machine" to "works, reliably, on infrastructure someone else can also understand," using the same rigor the rest of this book has applied to data: nothing asserted that hasn't been checked.

In plain English

Think about Riverstone opening a second warehouse in another city.

- **Cloud regions** are like choosing which city to build in: closer to your customers is faster, and rules differ by location.
 - **The shared responsibility model** is like renting a unit in a business park: the landlord secures the building, the fence, and the road; you lock your own door and decide who gets a key.
 - **Identity and access** is the list of who holds which keys: the delivery driver's key opens the loading bay, not the accounts office.
 - **A container** is a warehouse's stock, packed identically every time, so it behaves the same wherever it's unloaded, rather than depending on what happens to already be in that particular building.
 - **Kubernetes** is the logistics coordinator who decides which building gets which containers, replaces one that's damaged, and adds more when a big order comes in.
 - **Infrastructure as Code** is a written building plan, not a verbal instruction to a contractor: anyone can read it, review it before it's built, and rebuild the exact same warehouse in a different city.
 - **CI/CD** is the checklist that runs automatically before anything ships: inspected, tested, and only then loaded onto the truck.
-

52.1 Cloud fundamentals

Regions and availability zones

A **region** is a geographic area a cloud provider operates in (Mumbai, Singapore, Frankfurt). Each region contains multiple **availability zones**: physically separate data centers with independent power and networking, close enough for fast connections between them but far enough apart that one flooding or losing power doesn't take out the others.

Two decisions follow directly:

- **Choose a region near your users and your data's legal home.** Riverstone's customers, staff, and (per Chapter 45's note on data protection law) most of its personal data belong in an Indian region.

- **Spread anything that must stay up across at least two availability zones.** A pipeline running in one zone goes down when that zone has a bad afternoon; the same pipeline split across two zones usually doesn't.

The shared responsibility model

Every major cloud provider draws the same line, in the same place: the provider secures the **cloud** (physical data centers, the hardware, the virtualization layer, and for managed services, the service software itself); you secure what you put **in** the cloud (your data, your access controls, your network rules, your code, and how you configure every one of the provider's services). A misconfigured storage bucket left open to the internet is never the provider's fault under this model, however it happened. Chapter 64 covers the governance side of this line in depth; this chapter is about not crossing it by accident.

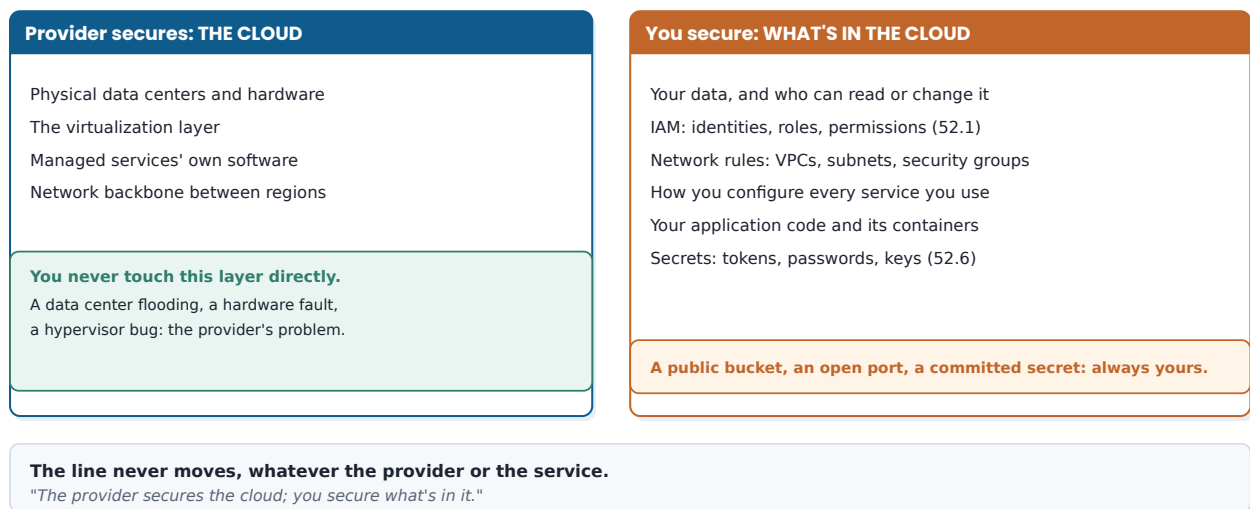


Figure 52.1 — The shared responsibility model draws one line, in the same place, at every cloud provider.

Identity and access management (IAM)

IAM controls who, or what, can do what. The habit that matters most is **least privilege**: grant exactly the permissions a task needs, and no more, and prefer a **role** a pipeline assumes over a permanent credential a person shares.

For Riverstone's pipeline, that means a dedicated identity that can read and write the sensor archive's bucket and nothing else, not an administrator's full access borrowed for convenience. Section 52.4's Terraform example creates exactly this: a role scoped to one bucket, with three specific actions.

Watch out: "it's just for now" access outlives its reason. Temporary broad permissions granted to unblock a deadline are the single most common source of cloud security incidents, because removing them is nobody's job once the deadline passes. Grant scoped access from the start; it costs a few extra minutes and no meetings six months later.

Networking: VPCs, subnets, and security groups

A **VPC** (virtual private cloud) is your own isolated slice of the provider's network. Inside it, **subnets** divide the address space into smaller ranges, usually one **public** (reachable from the internet, for things like a load balancer) and one **private** (not directly reachable, for a pipeline and a database) per availability zone. A **security group** is a firewall attached to a resource, allowing only the specific traffic it needs.

Subnets are sized in **CIDR notation**: an address range and a prefix length that says how many addresses it contains. This isn't a diagram to memorize; it's arithmetic you can check.

```
import ipaddress
from networking_math import describe

vpc = ipaddress.ip_network("10.0.0.0/16")
print(f"VPC {vpc}: {vpc.num_addresses:,} total addresses")

subnets = list(vpc.subnets(new_prefix=24))
print(f"split into /24 subnets: {len(subnets)} subnets of {subnets[0].num_addresses} addresses each")

plan = {
    "public-a (load balancer, Mumbai AZ-a)": subnets[0],
    "public-b (load balancer, Mumbai AZ-b)": subnets[1],
    "private-a (pipeline, database, AZ-a)": subnets[2],
    "private-b (pipeline, database, AZ-b)": subnets[3],
}
for name, subnet in plan.items():
    d = describe(str(subnet))
    print(f" {name} {d['cidr']:<12} usable hosts: {d['usable_hosts']}")
print(f"\nsubnets allocated: {len(plan)}, remaining for future use: {len(subnets) - len(plan)}")
```

```
VPC 10.0.0.0/16: 65,536 total addresses
split into /24 subnets: 256 subnets of 256 addresses each
  public-a (load balancer, Mumbai AZ-a) 10.0.0.0/24 usable hosts: 251
  public-b (load balancer, Mumbai AZ-b) 10.0.1.0/24 usable hosts: 251
  private-a (pipeline, database, AZ-a)   10.0.2.0/24 usable hosts: 251
  private-b (pipeline, database, AZ-b)   10.0.3.0/24 usable hosts: 251

subnets allocated: 4, remaining for future use: 252
```

Reading it. A `/16` network holds 65,536 addresses. Splitting it into `/24` subnets, each holding 256 addresses, gives 256 of them, of which four are allocated here: two public (for anything reachable from outside) and two private (for the pipeline and its database), one of each per availability zone, matching the "spread across zones" rule above. Cloud providers reserve five addresses per subnet for their own use (the network address, a router address, DNS, future use, and the broadcast address), leaving 251 usable hosts in each `/24`, far more than Riverstone's pipeline will ever need.

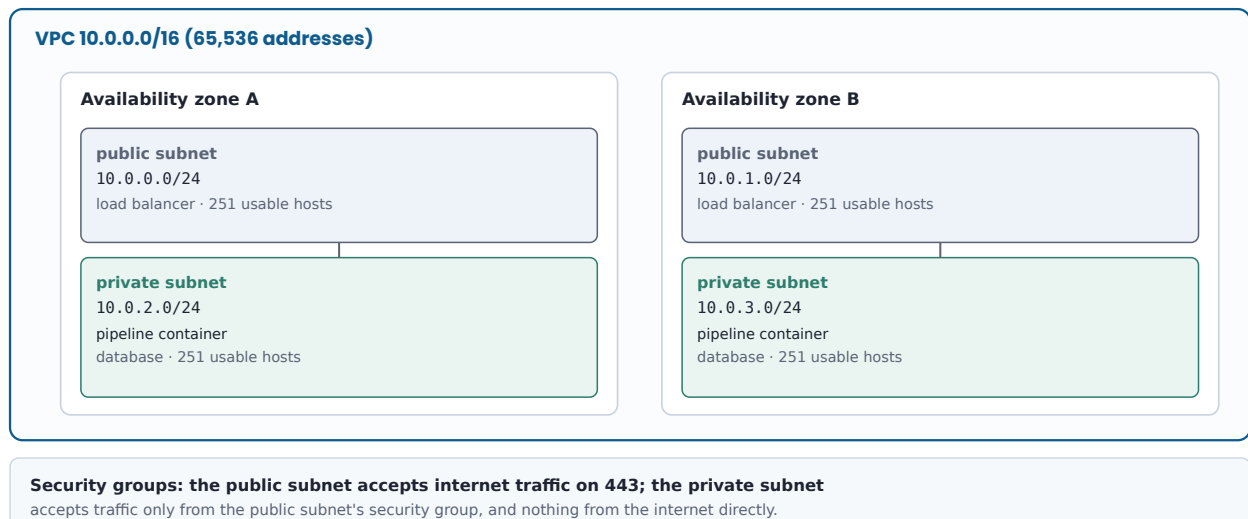


Figure 52.2 — Riverstone's VPC, matching the four subnets computed in this section: one public and one private subnet per availability zone.

The rule for security groups, expressed the way Chapter 47 expressed data tests: **default to denying everything, and add only the specific rule a real need justifies**. The pipeline's security group allows inbound traffic on its one port, from the load balancer's security group specifically, not from `0.0.0.0/0` (every address on the internet). Riverstone's database's security group allows inbound traffic on its port from the pipeline's security group, and nothing else.

52.2 Containers

Why containers exist

"It works on my machine" is a symptom of one thing: the code depends on details of that machine, whether library versions, environment variables, or configuration files, that aren't written down anywhere. A **container** packages an application with everything it needs to run, as a single, versioned artifact that behaves identically wherever it's run.

An **image** is the packaged artifact; a **container** is a running instance of that image. You build an image once and run as many containers from it as you need.

Building a Dockerfile for the Riverstone pipeline

A **Dockerfile** is a script that builds an image, one instruction at a time. Here's one for Chapter 46's pipeline, built up the way you'd actually write it:

```

FROM python:3.12-slim AS base

# System packages the pipeline needs at run time (nothing here is needed only for building)
RUN apt-get update \
    && apt-get install -y --no-install-recommends libpq-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Install dependencies before copying code: this layer only rebuilds when requirements.txt changes,
# not every time application code changes (this section's caching lesson).
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Now copy the application code
COPY ingest.py apply_day.py make_files.py mock_crm_api.py ./
COPY dagster_definitions.py .

# Never bake secrets into an image (section 52.6): the CRM token arrives at run time.
ENV RIVERSTONE_SOURCE="dbname=riverstone_source"
ENV PYTHONUNBUFFERED=1

# Run as a non-root user: a compromised container process shouldn't have root inside the image
RUN useradd --create-home pipeline
USER pipeline

EXPOSE 3000
CMD ["dagster", "dev", "-f", "dagster_definitions.py", "-h", "0.0.0.0", "-p", "3000"]

```

Every line, and why it's in that order.

- `FROM python:3.12-slim` starts from a small, official Python image rather than a full operating system: less to download, less surface to secure.
- The `apt-get install` line installs `libpq-dev`, which `psycopg2` needs to compile against, and cleans up the package list cache in the same instruction, so it isn't left behind in the image's layers.
- `COPY requirements.txt .` then `RUN pip install` happens **before** `COPY` of the application code. **Order matters for caching:** Docker builds in layers and reuses a cached layer if nothing that affects it has changed. If the code were copied first, editing a single line of `ingest.py` would invalidate every layer after it, including the slow dependency install. Copying dependencies first means changing application code rebuilds in seconds, not minutes.
- `ENV RIVERSTONE_SOURCE=...` sets a default that isn't secret. The CRM token, which **is** secret, is never set here; section 52.6 covers where it actually comes from.
- `RUN useradd` and `USER pipeline` mean the process inside the container runs as an ordinary user, not root, so a vulnerability in the application can't trivially escalate to control of the whole container.
- `CMD` is what runs when a container starts from this image: Dagster's development server, listening on all interfaces so it's reachable from outside the container.

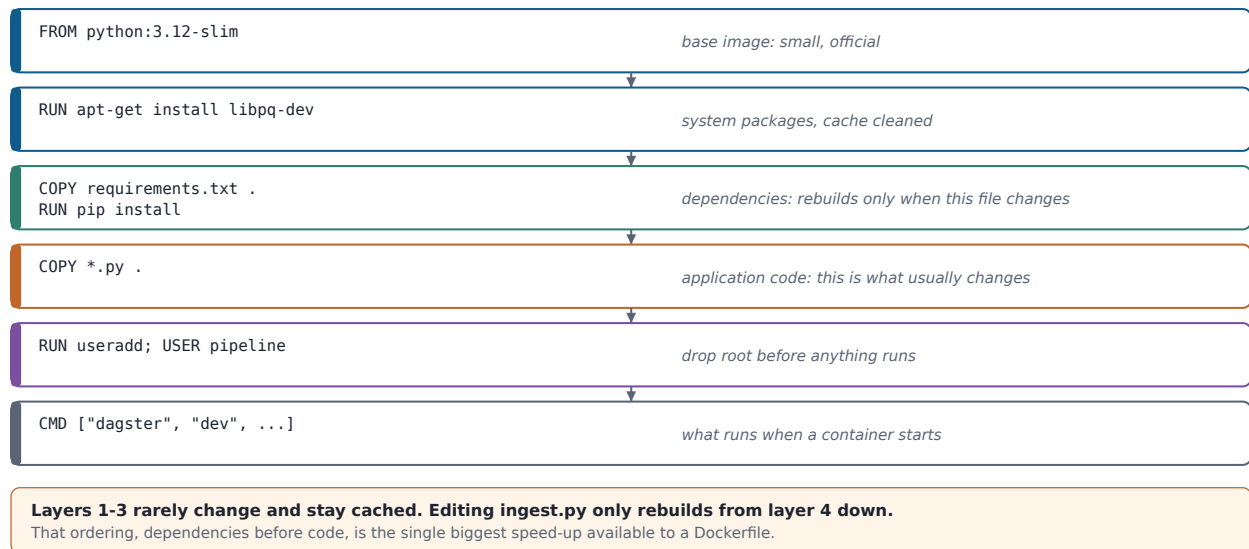


Figure 52.3 – The Dockerfile's layers, in build order. Ordering dependencies before code is what makes rebuilds fast.

Checking it's a valid image, for real

```
from dockerfile_parse import DockerfileParser

d = DockerfileParser(fileobj=open("pipeline_image/Dockerfile", "rb"))
print("base image      :", d.baseimage)
print("instructions used:", sorted(set(s["instruction"] for s in d.structure)))
print("layers (RUN/COPY/ADD):", sum(1 for s in d.structure if s["instruction"] in ("RUN", "COPY", "ADD")))
print("runs as user    :", [s["value"] for s in d.structure if s["instruction"] == "USER"])
```

```
base image      : python:3.12-slim
instructions used: ['CMD', 'COMMENT', 'COPY', 'ENV', 'EXPOSE', 'FROM', 'RUN', 'USER', 'WORKDIR']
layers (RUN/COPY/ADD): 6
runs as user    : ['pipeline']
```

Reading it. A real Dockerfile parser confirms the base image, every instruction used, and that six layers (the `RUN` and `COPY` instructions) will be built, and that the container drops root privilege before running anything. This is exactly what a linter would check before the image is ever built; **building** it, which needs a container runtime and a network path to pull the base image, isn't done here.

Running more than one container together

A single pipeline usually needs a database alongside it during development. **Docker Compose** describes a small stack of containers as one file:

```

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"      # from a .env file, never committed (section
52.6)
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  pipeline:
    build: .
    depends_on:
      - postgres
    environment:
      RIVERSTONE_SOURCE: "dbname=riverstone_source host=postgres user=postgres
password=${POSTGRES_PASSWORD}"
      CRM_API_TOKEN: "${CRM_API_TOKEN}"
    ports:
      - "3000:3000"
    restart: unless-stopped

volumes:
  pgdata:

```

```

import yaml

compose = yaml.safe_load(open("pipeline_image/docker-compose.yml"))
print("valid YAML, services:", list(compose["services"].keys()))
print("pipeline depends on :", compose["services"]["pipeline"]["depends_on"])
print("pipeline reads secrets from environment:", list(compose["services"]["pipeline"]
["environment"].keys()))

```

```

valid YAML, services: ['postgres', 'pipeline']
pipeline depends on : ['postgres']
pipeline reads secrets from environment: ['RIVERSTONE_SOURCE', 'CRM_API_TOKEN']

```

Reading it. Valid YAML, two services, and the pipeline correctly depends on the database starting first. Both secrets arrive through environment variables, referencing a `.env` file, never written into this file or the image itself.

52.3 Orchestrating containers

What Kubernetes actually solves

Running one container by hand is easy. Running many, across many machines, so that a crashed one restarts, traffic reaches a healthy one, and a busy period gets more capacity automatically, is what **Kubernetes** does. It's worth understanding even for a company Riverstone's size, because it's the vocabulary most cloud job descriptions and most managed platforms are built on.

Object	What it is
Pod	The smallest deployable unit: one or more containers that always run together, on the same machine
Deployment	A declaration of how many copies (replicas) of a Pod should exist, and how to roll out a new version
Service	A stable network address in front of a changing set of Pods, so other things can find them reliably
Secret	A piece of sensitive configuration (a token, a password), stored separately from the container image
Namespace	A way of dividing one cluster into separate areas, for different teams or environments

A manifest for the pipeline

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: riverstone-pipeline
  labels:
    app: riverstone-pipeline
spec:
  replicas: 2
  selector:
    matchLabels:
      app: riverstone-pipeline
  template:
    metadata:
      labels:
        app: riverstone-pipeline
    spec:
      containers:
        - name: pipeline
          image: riverstone/pipeline:1.4.0
          ports:
            - containerPort: 3000
          env:
            - name: RIVERSTONE_SOURCE
              valueFrom:
                secretKeyRef:
                  name: riverstone-db-secret
                  key: connection-string
            - name: CRM_API_TOKEN
              valueFrom:
                secretKeyRef:
                  name: riverstone-crm-secret
                  key: token
      resources:
        requests:
          cpu: "250m"
          memory: "512Mi"
        limits:
          cpu: "1"
          memory: "1Gi"
      readinessProbe:
        httpGet:
          path: /health
          port: 3000
        initialDelaySeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: riverstone-pipeline
spec:
  selector:
    app: riverstone-pipeline
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP
```

```
docs = list(yaml.safe_load_all(open("k8s/pipeline-deployment.yaml")))
print("documents in file:", len(docs))
for doc in docs:
    print(" ", doc["kind"], "- ", doc["metadata"]["name"])

deployment, service = docs
container = deployment["spec"]["template"]["spec"]["containers"][0]
print("\nreplicas      :", deployment["spec"]["replicas"])
print("cpu request/limit:", container["resources"]["requests"]["cpu"], "/", container["resources"]
["limits"]["cpu"])
print("secrets used      :", [e["valueFrom"]["secretKeyRef"]["name"] for e in container["env"]])
print("service exposes   : port", service["spec"]["ports"][0]["port"], "-> container port",
      service["spec"]["ports"][0]["targetPort"], f"({service['spec']['type']})")
```

```
documents in file: 2
Deployment - riverstone-pipeline
Service - riverstone-pipeline

replicas      : 2
cpu request/limit: 250m / 1
secrets used   : ['riverstone-db-secret', 'riverstone-crm-secret']
service exposes : port 80 -> container port 3000 (ClusterIP)
```

Reading it, and why each piece is there.

- **Two documents in one file**, separated by `---`: a Deployment and a Service, exactly as parsed.
- **replicas: 2** means two copies run at once; if one crashes, Kubernetes starts a replacement, and traffic keeps flowing to the other.
- **Secrets are referenced, never inlined**. Both environment variables pull from Kubernetes Secret objects by name, which is itself only a partial answer (a Secret is base64-encoded, not encrypted, by default); section 52.6 covers doing this properly.
- **resources.requests and resources.limits** tell the scheduler how much CPU and memory to reserve, and cap how much a runaway process can consume: the request (250m CPU, a quarter of a core) is what's guaranteed, and the limit (1 full core) is the ceiling.
- **readinessProbe** stops traffic reaching a container before it's actually ready to serve, which matters most in the seconds right after a deployment.
- **The Service** gives the two Pods one stable address (`riverstone-pipeline` , port 80) that doesn't change as Pods are replaced, forwarding to whichever Pod is healthy on the container's actual port 3000.

When you don't need it

Kubernetes solves problems that come with scale and many services. Riverstone, with one pipeline, doesn't have that problem yet. A single managed container service (Chapter 46's job, deployed as one container on a managed platform) is simpler, cheaper, and has almost nothing to operate. **Reach for Kubernetes when you have several services that need to be scheduled, scaled, and networked together**, not because the term appears in job postings. Section 52.8 returns to this choice directly.

52.4 Infrastructure as Code

Clicking through a cloud provider's console to create a bucket works once. It doesn't tell the next person what was created, why, or how to make an identical one in another region. **Infrastructure as Code (IaC)** describes infrastructure in files that can be reviewed, versioned, and applied repeatably, the same discipline Chapter 29 taught for application code.

Terraform is the most widely used tool for this, and it works with every major cloud provider through the same workflow: write configuration, run `terraform plan` to see exactly what would change, and `terraform apply` to make it happen. Terraform keeps a **state file** recording what it created, which is how it knows the difference between "create this" and "this already exists, leave it alone."

Here's a configuration for the storage behind Chapter 49's sensor archive, including the identity that's allowed to touch it:

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  description = "AWS region for Riverstone's data platform"
  type        = string
  default     = "ap-south-1"          # Mumbai, closest region to Riverstone's operations
}

variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "production"
}

resource "aws_s3_bucket" "sensor_archive" {
  bucket = "riverstone-sensor-archive-${var.environment}"

  tags = {
    Project      = "analyst-to-architect"
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}

resource "aws_s3_bucket_versioning" "sensor_archive" {
  bucket = aws_s3_bucket.sensor_archive.id
  versioning_configuration {
    status = "Enabled"          # protects against the Chapter 49 "table nobody could fix"
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "sensor_archive" {
  bucket = aws_s3_bucket.sensor_archive.id
  rule {
    id      = "archive-old-versions"
    status  = "Enabled"
    noncurrent_version_transition {
      noncurrent_days = 90
      storage_class   = "GLACIER"    # cold storage for anything older than 90 days (Chapter 49, section 49.8)
    }
  }
}

resource "aws_iam_role" "pipeline_role" {
  name = "riverstone-pipeline-${var.environment}"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "ecs-tasks.amazonaws.com" }
    }]
  })
}

```

```

    })
}

resource "aws_iam_role_policy" "pipeline_s3_access" {
  name = "sensor-archive-read-write"
  role = aws_iam_role.pipeline_role.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = ["s3:GetObject", "s3:PutObject", "s3:ListBucket"]
      Resource = [
        aws_s3_bucket.sensor_archive.arn,
        "${aws_s3_bucket.sensor_archive.arn}/*"
      ]
    }]
  })
}

output "bucket_name" {
  value = aws_s3_bucket.sensor_archive.bucket
}

```

```

import hcl2

with open("terraform/main.tf") as f:
    cfg = hcl2.load(f)

resource_types = [list(r.keys())[0].strip(' ') for r in cfg["resource"]]
variables = [list(v.keys())[0].strip(' ') for v in cfg["variable"]]
outputs = [list(o.keys())[0].strip(' ') for o in cfg["output"]]
print("valid HCL, top-level blocks:", list(cfg.keys()))
print("resources declared:", resource_types)
print("variables declared:", variables)
print("outputs declared  :", outputs)

```

```

valid HCL, top-level blocks: ['terraform', 'provider', 'variable', 'resource', 'output',
'__comments__']
resources declared: ['aws_s3_bucket', 'aws_s3_bucket_versioning',
'aws_s3_bucket_lifecycle_configuration', 'aws_iam_role', 'aws_iam_role_policy']
variables declared: ['aws_region', 'environment']
outputs declared  : ['bucket_name']

```

Reading it, resource by resource.

- **aws_s3_bucket** creates the storage, named from a variable so the same file builds a `staging` or `production` bucket without editing anything.
- **aws_s3_bucket_versioning** turns on the safeguard that would have prevented Chapter 49's incident: every version of every object is kept, so a bad overwrite can be undone.
- **aws_s3_bucket_lifecycle_configuration** automatically moves old versions to cold storage after 90 days, exactly the retention-and-cost trade-off Chapter 49's section 49.8 worked through.
- **aws_iam_role** and **aws_iam_role_policy** are section 52.1's least-privilege rule, written as code: this identity can `GetObject`, `PutObject`, and `ListBucket`, on this one bucket, and nothing else. Nobody has to remember to scope it correctly by hand; it's the only thing this file allows.
- **output "bucket_name"** prints the created bucket's name after `apply`, so other configurations (or a human) can reference it without hunting through a console.

Plan, apply, and why review matters

- **terraform plan** shows what would change, without changing anything: resources to add, modify, or destroy. Reading a plan carefully, especially any line that says **destroy**, is the single habit that prevents most infrastructure incidents.
- **terraform apply** makes the change, after asking for confirmation (or automatically in CI, once a plan has been reviewed).
- **State** must be stored somewhere shared and locked (a cloud storage bucket with locking, not a laptop), or two people running Terraform at once can corrupt each other's changes.
- **Every change goes through a pull request**, reviewed like any other code change, precisely because "it's just infrastructure" is how the closing story in this chapter goes wrong.

Simplification note. This configuration is written to be read and understood, and its syntax is verified. It has never been run against a real AWS account in producing this book, for the reasons explained at the top of the chapter. Treat it as a correct starting point to adapt, not as a file to copy and apply unread.

52.5 CI/CD

Continuous integration runs checks (tests, linting) automatically on every change. **Continuous deployment** (or delivery) takes a change that passes those checks and ships it, automatically or with one approval. Together, **CI/CD** is what makes "every change is reviewed and tested before it reaches production" true by construction, rather than by everyone remembering to do it.

Here's a workflow for Riverstone's pipeline using **GitHub Actions**, one of the most common CI/CD tools:

```

name: Riverstone pipeline CI/CD

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r pipeline_image/requirements.txt pytest
      - run: pytest tests/

  build-and-push:
    needs: test
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t riverstone/pipeline:${{ github.sha }} pipeline_image/
      - name: Push image
        run: docker push riverstone/pipeline:${{ github.sha }}

  deploy:
    needs: build-and-push
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Update deployment
        run: kubectl set image deployment/riverstone-pipeline pipeline=riverstone/pipeline:${{ github.sha }}

```

Working out exactly what runs, for real

Reading a workflow file and knowing what it will actually do for a given event is a skill worth having independently of any specific tool. Here it's checked properly: the same YAML, loaded and evaluated against GitHub's own trigger and condition rules, written out in Python.

```

from simulate_workflow import load_workflow, jobs_that_run

wf = load_workflow(".github_workflows/deploy.yml")
cases = [
    ("pull_request", "main", "a pull request targeting main"),
    ("push", "main", "a direct push to main"),
    ("push", "develop", "a push to a branch other than main"),
]
for event, branch, description in cases:
    print(f"{description:<38} -> {jobs_that_run(wf, event, branch)}")

```

```

a pull request targeting main      -> ['test']
a direct push to main             -> ['test', 'build-and-push', 'deploy']
a push to a branch other than main -> []

```

Reading it, and why each result is correct.

- A **pull request targeting main** triggers the workflow (the `pull_request` rule matches), so `test` runs. `build-and-push` has `if: github.event_name == 'push' && ...`, which is false for a pull request, so it, and `deploy` after it, don't run. This is exactly right: **you want tests to run on every proposed change, and deployment to happen only once it's merged.**
- A **direct push to main** matches the `push` trigger, `test` runs, its condition is satisfied so `build-and-push` runs, and `deploy`'s condition is satisfied too: all three run in order, because `needs:` makes each wait for the one before it to succeed.
- A **push to any other branch** matches nothing in the `on:` block at all: the workflow doesn't start.

What each job does, and what isn't executed here. `test` installs the pipeline's dependencies and runs its test suite, the same tests Chapter 29 taught you to write, and this is ordinary Python that could genuinely run in CI. `build-and-push` and `deploy` need a real container registry and a real cluster, so they're shown as GitHub Actions would run them, not executed as part of writing this book.

What good CI/CD adds beyond "it runs"

- **Fail fast, fail clearly.** A failed test should stop the pipeline from deploying, with a message that says what broke, not a general failure.
- **Environments with protection rules.** The `environment: production` line can require a named approver before that job runs, which is where a human checkpoint belongs for the highest-stakes step.
- **Rollback is a redeploy, not a rescue mission.** If `deploy` points at an image tagged by commit (`{{ github.sha }}`), rolling back is deploying the previous commit's tag, a normal, tested action rather than an emergency improvisation.

52.6 Secrets, done properly

Chapter 45 said tokens don't belong in code. Here's the full version of where they do belong, tying together everything this chapter has built:

Layer	Where the secret lives	Not here
Local development	A <code>.env</code> file, listed in <code>.gitignore</code> , never committed	Hardcoded in a script
Docker Compose	Read from the shell environment or a <code>.env</code> file, referenced by <code>\${VARIABLE}</code>	Baked into the image with <code>ENV</code>
Kubernetes	A Secret object (better: sealed or provider-managed, since raw Secrets are only base64-encoded)	A ConfigMap, or committed YAML
CI/CD	The platform's own encrypted secrets store (GitHub Actions secrets, and similar in every CI tool)	A workflow file, even a private one
Cloud infrastructure	A managed secrets service (AWS Secrets Manager, Azure Key Vault, Google Secret Manager)	A Terraform variable's default value

The common failure across every layer is the same: a secret that was "just for testing" ends up committed to Git, and Git remembers everything, including in a repository's history, forever, even after the file is deleted. **Rotate a leaked credential immediately, and treat "leaked to the wrong channel" the same as "leaked publicly."** Chapter 47's incident process applies directly: detect, classify severity, contain by rotating, fix, review.

52.7 Cost

Chapter 49 estimated the sensor archive's storage cost. Compute, and how you run it, changes the total more than storage usually does.

```
from cost_estimate import container_compute_cost, always_on_vs_scheduled, HOURS_PER_MONTH

always_on, scheduled = always_on_vs_scheduled()
print(f"pipeline container, always on ({HOURS_PER_MONTH} h/month): ${always_on}/month")
print(f"same container, scheduled (~1 h/day, 30 h/month)      : ${scheduled}/month")
print(f"ratio: {round(always_on / scheduled, 1)}x more expensive to run it always on")

storage_month = round(277 * 0.023, 2)          # Chapter 49's sensor-archive figure
total = round(storage_month + scheduled, 2)
print(f"\nstorage (Chapter 49)           : ${storage_month}/month")
print(f"compute (scheduled pipeline) : ${scheduled}/month")
print(f"total estimate                   : ${total}/month")
```

```
pipeline container, always on (730 h/month): $17.52/month
same container, scheduled (~1 h/day, 30 h/month)      : $0.72/month
ratio: 24.3x more expensive to run it always on

storage (Chapter 49)           : $6.37/month
compute (scheduled pipeline) : $0.72/month
total estimate                   : $7.09/month
```

Reading it. A container running around the clock costs about **24 times more** than the same container running only for the hour or so the pipeline actually needs each day. This is the same lesson as Chapter 49's storage-scanning arithmetic, applied to compute: **the cheapest infrastructure decision is usually "don't run it when nothing needs it,"** not a smaller instance type. Scheduled or serverless compute (a container that starts on a trigger and stops when it's done, Chapter 46's job model exactly) is very often the right default for a batch pipeline, with an always-on service reserved for things that must answer requests at any moment, such as a webhook receiver from Chapter 51.

Total estimate for Riverstone's pipeline and its sensor archive, storage and compute together: **about \$7 a month**. It's a small number for a real reason: most of what this book has built runs briefly, once a day, on a modest amount of data. That won't stay true forever, and knowing how to estimate it, rather than guessing, is what lets you notice when it stops being true.

52.8 Choosing a deployment target

Option	Fits	Operational cost
A single managed container service (AWS App Runner, Google Cloud Run, Azure Container Apps)	One or a few independent services, like Riverstone's pipeline today	Very low: little to configure, scales automatically within limits
A managed orchestrator (AWS ECS, Google Cloud Run for services with more control)	A handful of services that need to talk to each other, without full Kubernetes complexity	Low to moderate
Managed Kubernetes (EKS, GKE, AKS)	Many services, teams, and workloads sharing a platform	Moderate to high: someone has to own cluster upgrades, networking, and access
Self-managed Kubernetes	Rarely justified below significant scale, or strict on-premises requirements	High: you own everything a managed service would have handled
Serverless functions (Lambda, Cloud Functions)	Short, event-triggered tasks (a webhook receiver, a small transform)	Very low, pay-per-invocation

For Riverstone today, a single managed container service, running on the schedule Chapter 46 already defined, is the right size: it deploys from the CI/CD workflow in section 52.5, costs a few dollars a month, and needs nobody dedicated to operating it. The Kubernetes manifest in section 52.3 is worth having written and understood, because the vocabulary is universal and the day this company runs ten services instead of one, the migration is a known, well-trodden path rather than a redesign from nothing.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Granting broad access "for now"	Permissions nobody remembers to remove	Scope access narrowly from the start (52.1)
A security group open to the whole internet	Anyone can reach a database or pipeline	Allow only the specific source that needs it
Copying application code before installing dependencies in a Dockerfile	Every build reinstalls everything, even for a one-line code change	Copy dependency files first, install, then copy code (52.2)
Running a container process as root	A vulnerability inside the container has full control of it	Create and switch to a non-root user
Reaching for Kubernetes for one service	Complexity with nothing to show for it	Use it when several services need scheduling together (52.3, 52.8)
Clicking through a console instead of writing IaC	Nobody can reproduce what exists, or review a change before it happens	Terraform (or equivalent), reviewed like code (52.4)
Applying Terraform without reading the plan	An unexpected <code>destroy</code> runs in production	Read every plan; treat <code>destroy</code> lines as a stop sign

Mistake	Symptom	Fix
Storing Terraform state on a laptop	Two people's changes corrupt each other	Shared, locked remote state
Secrets in a committed file, "just this once"	The credential is in Git history forever, even after deletion	A secrets store at every layer (52.6); rotate immediately if it happens
Running compute around the clock for a job that runs once a day	A bill many times larger than necessary	Scheduled or serverless compute for batch work (52.7)
No environment protection on the deploy step	A merge to main ships straight to production with no checkpoint	Required approval on the production environment (52.5)

In the real world: the afternoon the warehouse went dark

Riverstone's contract data engineer was cleaning up the Terraform configuration for the sensor archive's networking, removing what looked like an unused security group rule. He ran `terraform apply`, saw the familiar list of small changes, and confirmed it without reading past the first few lines.

Twenty minutes later, Meera couldn't load the sales dashboard. The pipeline's nightly Flash hadn't sent. The engineer's "unused" rule was in fact the one allowing the pipeline's container to reach the database at all; a previous engineer had added it directly through the cloud console, months earlier, and it had never been added to the Terraform file. To Terraform, a rule it didn't know about looked exactly like a rule that shouldn't exist, and the plan had said so, on a line nobody had scrolled down to see.

Restoring it took ten minutes once the cause was found, and finding the cause took two hours, because nothing had logged **why** connectivity had disappeared, only that it had.

The fixes were the ones this chapter has built toward. Every resource that existed was **imported** into Terraform, so the file and reality agreed completely, with no manual console changes left unrecorded anywhere. Every `apply` after that went through a pull request, with the **plan's full output posted for review**, not just a summary. And a Chapter 47-style check was added: a scheduled test that tries a real database connection from a throwaway container and alerts within minutes if it fails, rather than waiting for a missed report to be the first sign.

At the retrospective, the engineer made the point himself before anyone else needed to. "The plan told me exactly what was about to happen. I didn't read it, because it looked routine."

What made this work.

- **The cause was a gap between reality and the code describing it**, not a bug in Terraform, which did precisely what it was told.
- **Reading the plan is the review step**, and skipping it is the same mistake as merging code no one looked at.
- **Importing everything into code** closed the gap for good, rather than patching this one incident.

- A **synthetic connectivity check** meant the next version of this failure would be caught in minutes, not discovered by a missing report.
-

Tools

- **Python 3.12** with `pyyaml`, `python-hcl2`, and `dockerfile-parse` for the validation shown in this chapter (`pip install pyyaml python-hcl2 dockerfile-parse`).
 - **The Chapter 52 companion folder** (`companion/ch52/`): `pipeline_image/` (Dockerfile, `requirements.txt`, `docker-compose.yml`), `k8s/pipeline-deployment.yml`, `terraform/main.tf`, `.github_workflows/deploy.yml`, `simulate_workflow.py`, `networking_math.py`, `cost_estimate.py`.
 - **For real use, when you have the services:** Docker Desktop or Docker Engine; `kubectl` and a cluster (a local one via `kind` or `minikube` is enough to learn on); Terraform CLI with a real cloud account; a GitHub repository for Actions to run in.
 - **Versions used for the checks shown:** Python 3.12.3, PyYAML 6.0.3.
-

The project: deploy Riverstone's pipeline

Goal: a complete, reviewed deployment package for the Chapter 46 pipeline: container, orchestration manifest, infrastructure as code, and a CI/CD workflow, validated as far as this book's tools allow, with an honest runbook for what remains to actually deploy it.

Option A: Riverstone. Extend the companion files.

Option B: your own project. Any application you've built in this book, containerized the same way.

Steps

1. **Write a Dockerfile** for a pipeline of your choice, ordered for caching, running as a non-root user, with no secrets baked in. Validate it with `dockerfile-parse`.
2. **Write a docker-compose.yml** for local development, with secrets from environment variables. Validate it as valid YAML and confirm its service dependencies.
3. **Write a Kubernetes Deployment and Service** for the same application. Validate the YAML, and write down, in your own words, what would happen if a Pod crashed.
4. **Write a small Terraform configuration** for one piece of infrastructure your project needs (storage, a role, a network rule). Validate it with `python-hcl2`, and write the IAM policy with the narrowest permissions you can justify.
5. **Write a CI/CD workflow** with at least a test job and a deploy job gated on the test passing and the branch being `main`. Simulate its trigger logic in Python for three events, as this chapter did.
6. **Write your secrets plan:** where each secret lives at each layer, referencing section 52.6's table.
7. **Estimate the monthly cost** of running it, using section 52.7's approach, and compare an always-on option with a scheduled one.

8. **Write the deployment runbook:** the exact steps a person would follow to deploy this for real, including what they'd check in the Terraform plan before approving it, and what they'd do if the deploy step failed.

Stretch goals

- If you have access to Docker, actually build the image and run the compose stack locally.
- If you have a free-tier cloud account, apply the Terraform configuration to a throwaway project and destroy it afterward, reading every line of both plans.
- Add a synthetic health check to your CI/CD workflow, in the style of this chapter's closing story.

You've got it when...

- ☐ You can explain regions, availability zones, and the shared responsibility model in your own words.
- ☐ You can design an IAM policy that grants the narrowest access a task needs.
- ☐ You can compute how many usable addresses a given CIDR range provides, and split a VPC into subnets by hand.
- ☐ You can explain why containers solve "it works on my machine", and the difference between an image and a container.
- ☐ You can write a Dockerfile ordered for caching, running as a non-root user.
- ☐ You can read a Kubernetes Deployment and Service and say what each field does.
- ☐ You can explain what Terraform's state file is for, and why you always read a plan before applying it.
- ☐ You can trace exactly which jobs in a CI/CD workflow run for a given event, using its actual trigger and `if` conditions.
- ☐ You can name where a secret should live at every layer, from a laptop to production.
- ☐ You can estimate the cost difference between always-on and scheduled compute.
- ☐ You can recommend a deployment target for a company of a given size, and say when Kubernetes earns its complexity.

Recap

- **Regions and availability zones** are about geography and independence; spread anything critical across at least two zones.
- The **shared responsibility model** draws one clear line: the provider secures the cloud, you secure what's in it.
- **Least privilege** IAM, and real **CIDR arithmetic** for networking: a `/16` VPC split into `/24` subnets gives 256 subnets of 251 usable hosts each.

- **Containers** package an application with everything it needs; **Dockerfiles** should install dependencies before copying code, and run as a non-root user.
- **Kubernetes** objects (Pod, Deployment, Service, Secret) solve scheduling, healing, and networking for many services; it's worth understanding even when you don't need it yet.
- **Infrastructure as Code**, checked with `terraform plan` before every `apply`, makes infrastructure reviewable, versioned, and reproducible, and prevents the "nobody knows why this exists" problem.
- **CI/CD** runs tests on every change and deploys only what passes; a workflow's trigger and `if` conditions can be traced precisely, as this chapter did for real.
- **Secrets** belong in a dedicated store at every layer, never in code, images, or committed files, and a leak anywhere means an immediate rotation.
- **Compute cost is dominated by whether something runs when nothing needs it**: the same container cost about 24 times more running constantly than running on Riverstone's actual schedule.
- **Choose the simplest deployment target that fits today's number of services**, and treat Kubernetes as a destination to grow into, not a default to start with.

Practice exercises

Warm-up

1. A cloud provider's storage bucket is left publicly readable by mistake. Whose responsibility is that under the shared responsibility model, and why?
2. A VPC is `10.20.0.0/16`. How many total addresses does it contain, and how many `/24` subnets can it be split into?
3. In a Dockerfile, which should come first: `COPY requirements.txt .` and `RUN pip install`, or `COPY . .` (copying all the application code)? Explain why in terms of build caching.

Core

4. A `/24` subnet is created for a database that will only ever need 40 addresses. Using the reservation rule from section 52.1, how many usable addresses does it actually get, and what does that suggest about right-sizing a subnet?
5. Using this chapter's CI/CD workflow, trace what happens for a pull request from a feature branch targeting a branch called `develop` (not `main`). Which jobs run, and why?
6. Explain the difference between a Kubernetes `resources.requests` value and a `resources.limits` value, using the pipeline manifest's CPU values (`250m` requested, `1` as the limit) as your example.
7. The Terraform example in section 52.4 grants the pipeline's role `s3:GetObject`, `s3:PutObject`, and `s3:ListBucket` on one bucket. Using section 52.1's least-privilege rule, explain why `s3:DeleteObject` and access to other buckets are deliberately left out.

8. A container is configured to run 24 hours a day for a pipeline that only needs to execute once daily for about an hour. Using the cost model in section 52.7, calculate the ratio in cost between the two approaches, and explain in one sentence why the ratio is that large.
9. In the closing story, `terraform plan` correctly showed the security group rule as something to be destroyed, and the engineer still caused an outage. What does this tell you about the limits of tooling alone, and what process change actually fixed it?

Stretch

- l0. Design the IAM setup for a second pipeline that needs to read from the sensor archive bucket but must never be able to write to it. Write the policy statement's `Action` and `Resource` fields the way section 52.4 did, and explain what would go wrong if you granted it the existing role instead of a new one.
- l1. Riverstone's pipeline grows from one service to four: ingestion, the Dagster web interface, a webhook receiver from Chapter 51, and a small internal dashboard. Using section 52.8, make the case for moving to a managed orchestrator or Kubernetes at this point, and say what you'd want to see before agreeing.
- l2. Write a GitHub Actions workflow (or extend this chapter's one) that adds a `staging` deployment on every push to a `staging` branch, separate from the existing `production` deployment on `main`. Trace, as this chapter did, which jobs would run for a push to `staging`.

Think about it (no code needed)

- l3. "Faking output would be worse than saying so plainly." Explain why a book that claims to have run code it hasn't would be more damaging to a reader than one that clearly labels what wasn't executed, using an example from earlier in this book (Chapters 46 or 50) if it helps.
- l4. A colleague argues that since the cloud is so cheap for a company Riverstone's size (a few dollars a month), cost discipline doesn't matter yet and can wait until the company is bigger. Argue against that position using this chapter's ideas.

Key terms

region · availability zone · shared responsibility model · IAM (identity and access management) · least privilege · role · VPC (virtual private cloud) · subnet · CIDR notation · security group · container · image · Dockerfile · layer · build cache · Docker Compose · Kubernetes · Pod · Deployment · replica · Service · Secret (Kubernetes) · namespace · resource requests and limits · readiness probe · Infrastructure as Code (IaC) · Terraform · provider · state file · plan · apply · CI/CD · continuous integration · continuous deployment · GitHub Actions · workflow trigger · job · secrets manager · serverless · managed container service

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 62, The Economics of Data Platforms**, and **Chapter 65, FinOps**, return to cost management at the level of an entire organization's cloud spend.
 - **Chapter 64, Security, Privacy, Governance & Responsible AI**, builds on the shared responsibility model and least-privilege access from section 52.1.
 - **Chapter 46's** pipeline, **Chapter 47's** checks, **Chapter 49's** storage, **Chapter 50's** streaming jobs, and **Chapter 51's** syncs are all things this chapter's infrastructure would actually run.
 - **Chapter 63, Designing Automation & Integration Architecture**, returns to choosing deployment targets at the whole-company scale this chapter's section 52.8 only began.
 - **Part VIII**: cloud, containers, and deployment questions appear in the data engineering interview chapters, and system design cases in Chapter 77 routinely ask how you'd deploy and operate exactly what this chapter builds.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. It's the **customer's** responsibility. Under the shared responsibility model, the provider secures the underlying infrastructure and, for a managed storage service, the software of the service itself, but **configuration of that service**, including who can read a bucket, is squarely the customer's job. "The provider secures the cloud; you secure what's in it" draws the line exactly here.
2. A `/16` has $32 - 16 = 16$ bits free, so $2^{16} = 65,536$ total addresses. Splitting into `/24` subnets uses $24 - 16 = 8$ more bits per subnet, so $2^8 = 256$ subnets.
3. **COPY requirements.txt . and RUN pip install should come first.** Docker caches each instruction's result and reuses it if nothing that affects it has changed. If `COPY . .` (all the code) came first, then editing any single application file would invalidate that layer and every layer after it, forcing a full dependency reinstall on every code change. Installing dependencies from a separate, rarely-changing file first means that layer stays cached across nearly every rebuild.
4. The subnet still gets a full `/24` of 256 addresses, minus the provider's 5 reserved addresses, leaving **251 usable addresses**, of which only 40 will ever be used. This suggests the subnet is oversized for its purpose: a smaller `/26` (64 addresses, 59 usable after reservations) would comfortably fit 40 hosts with room to grow, leaving far more of the address space free for other subnets rather than reserving a needlessly large range for a small, fixed need.
5. **Nothing runs.** The workflow's `on: block` only triggers on `pull_request` targeting `main` and push to `main`. A pull request targeting `develop` doesn't match the `pull_request` trigger's `branches: [main]` restriction at all, so the workflow never starts, and none of `test`, `build-and-push`, or `deploy` run.

6. **requests** is what Kubernetes guarantees will be available to the container and uses when deciding which machine to schedule it on: `250m` means a quarter of one CPU core is reserved for it. **limits** is the ceiling the container is never allowed to exceed: `1` means it can burst up to a full core under load, but no further, even if more is available on the machine. A container using more than its request but less than its limit is normal; one hitting its limit gets throttled (for CPU) or killed and restarted (for memory).

7. Granting only `GetObject`, `PutObject`, and `ListBucket` on this one bucket means the role can read and write sensor data and see what's in the bucket, which is everything the pipeline actually does. Leaving out `s3:DeleteObject` means a bug in the pipeline, or a compromised credential, **cannot delete data**, only add to it, which combined with the bucket's versioning (also in this configuration) makes accidental or malicious data loss far harder. Leaving out access to other buckets means even a fully compromised pipeline credential can't touch Riverstone's other cloud storage, containing the blast radius of any single failure to exactly the one resource that needed access, which is least privilege in practice, not just in principle.

8. From the worked figures, always-on costs \$17.52/month and scheduled costs \$0.72/month, a ratio of about **24 times**. The ratio is that large because the container's cost is proportional to the **hours it runs**, and an hour-a-day job uses roughly 30 of the month's 730 hours: paying for the other 700 idle hours is nearly pure waste for a batch job with no reason to be listening for requests around the clock.

9. It shows that **tooling correctly informing you is not the same as the information being acted on**: `terraform plan` did exactly its job, and the outage still happened, because the output wasn't read carefully. The process change that actually fixed it wasn't a better tool, it was **making the plan's full output part of a reviewed pull request**, so a second person, not under the time pressure of "just tidying up," had a chance to notice the destructive line before it was applied. Tools reduce the chance of an unreviewed mistake; only a review step catches the mistake a tool correctly reported and a person missed.

10. A sample policy statement: `Action = ["s3:GetObject", "s3:ListBucket"]`, `Resource = [aws_s3_bucket.sensor_archive.arn, "${aws_s3_bucket.sensor_archive.arn}/*"]`, a separate `aws_iam_role` from the existing pipeline role. Granting the **existing** role instead would give this new, read-only pipeline the same `PutObject` write access as the original pipeline, which violates least privilege for no reason: if this second pipeline is ever compromised or has a bug, it would be able to write or corrupt data it was only ever supposed to read, an entirely avoidable risk once you've noticed it needs a distinct, narrower role.

11. The case for moving up: with **four services** that need independent scaling, restarting, and internal networking (the dashboard talking to the pipeline's API, say), a single managed container service starts to mean four separate deployments to coordinate by hand, with no shared service discovery, health checking, or rollout strategy between them, exactly the coordination problem Kubernetes (or a lighter managed orchestrator like ECS) exists to solve. Before agreeing, you'd want to see: that the four services genuinely need to talk to each other (if they're fully independent, four separate simple deployments may still be simpler than one shared platform); that someone is willing to own cluster upgrades and access control ongoing, not just the initial setup; and a cost comparison, since a managed orchestrator or cluster typically has a higher baseline cost than four small independent services, which only pays off once the coordination problem is real.

12. A sample extension adds a new job block gated on the staging branch, for example a `deploy-staging` job with `if: github.event_name == 'push' && github.ref == 'refs/heads/staging'`, needing a renamed or separate `build-and-push` step (or a parallel one) so the two environments don't collide, and its own `environment: staging` for protection rules appropriate to a lower-stakes environment (perhaps no required approval, unlike production). Tracing a push to `staging`: it wouldn't match the existing `on: push: branches: [main]` trigger at all unless `staging` is added to that list; once added, `test` would run (assuming no branch restriction on it), and only the new `deploy-staging`-related jobs would satisfy their `if` conditions, while the original `production`-bound `build-and-push` and `deploy` jobs, still conditioned on `refs/heads/main`, would correctly not run.

13. A book that quietly presents fabricated output as if it were real teaches the reader a false picture of what actually happens when code runs, which is far worse than a gap honestly marked, because the reader has no way to know which parts to distrust. Chapter 46 marked its `run_failure_sensor` and Airflow examples as not executed, and Chapter 50 did the same for its Kafka client code, precisely so a reader building on this book's examples knows exactly which parts have been proven to work as shown and which are correct-looking illustrations still needing their own testing in a real environment. Once a single fabricated output is discovered, a reader reasonably starts doubting every other output in the book, even the ones that were genuinely verified, which destroys far more trust than the honest gap ever would.

14. Three things from this chapter argue against waiting. First, **habits formed on a small bill are the habits that scale**: a team that never learns to distinguish always-on from scheduled compute, or to read a cost estimate before deploying, doesn't suddenly develop that discipline when the bill crosses some threshold, they carry the same unexamined choices into a much larger number. Second, the **24-times cost difference** in section 52.7 between always-on and scheduled compute is a *ratio*, not an absolute; it applies exactly as much at Riverstone's current scale as it will at ten times the scale, and the sooner the right default is in place, the more total waste is avoided across the company's whole future, not just today's few dollars. Third, cost estimation is itself a skill, exactly like the reconciliation habits Chapter 47 taught for data: the point of estimating early isn't the money saved on a small bill, it's building the muscle of noticing *when something has stopped being small*, which only works if you've been checking all along, not starting the habit the day the number becomes alarming.